

Introduction to Data Structures

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

Recap of Lists and Tuples

Lists and Tuples are the first data structures we saw in this class
We call them data structures because they store data in a structured way!

- **Lists are created by surrounding a comma separated sequence with brackets [] !**
Recall that lists can be modified!
- **Tuples are created by surrounding a sequence with parentheses () !**
Recall that tuples cannot be modified!

```
1 numbers: list[int] = [1, 3, 3, 7]
2 letters: tuple[str] = ("a", "c", "b", "a", "z")
3
4 print(numbers)
5 print(letters)
```

```
[1, 3, 3, 7]
```

Recall Lists and Tuple Ordering

Lists and Tuples store things in an ordered fashion using indices!
Because individual elements are positioned, they can also store duplicates!

[1 , 3 , 3 , 7]

("a" , "c" , "b" , "a" , "z")

0

1

2

3

0

1

2

3

4

```
1 numbers: list[int] = [1, 3, 3, 7]
2 letters: tuple[str] = ("a", "c", "b", "a", "z")
3
4 print(numbers[0], numbers[1:], ": numbers has", len(numbers), "elements!")
5 print(letters[1], letters[2:], ": letters has", len(letters), "elements!")
```

```
1 [3, 3, 7] : numbers has 4 elements!
```

Data Structure Traits

There are three traits that differentiate data structures:

- **Ordered / Semi-ordered / Unordered**

This trait determines how the data within the structure is maintained

- **Mutable / Unchangeable / Immutable**

This trait determines if / how the data within the structure can be changed

- **Supports Duplicates / Does not support Duplicates**

This trait determines if the structure is capable of storing duplicates

Ordered / Semi-ordered / Unordered

Determines how the data within the structure is maintained!

This trait supports three different options:

-  **Ordered**
The data within the structure is strictly ordered in a consistent way
-  **Semi-ordered**
Some of the data within the structure is strictly ordered in a consistent way, other aspects of the data may be inconsistent or change over time
-  **Unordered**
The data within the structure is inconsistent / unreliable and changes over time

Mutable / Unchangeable / Immutable

Determines if the data within the structure can be changed

This trait supports three different options:

-  **Mutable**
Data can be added to, changed, or removed from the structure freely
-  **Unchangeable**
Data can be added to or removed from the structure freely but cannot be changed while contained within the structure
-  **Immutable**
The data within the structure is fixed after creation - new data cannot be added and existing data cannot be removed or modified

Supports Duplicates / Does not support Duplicates

Determines if the structure is capable of storing duplicates

This trait supports two different options:

-  **Supports Duplicates**
The structure can store exact copies of the same elements
-  **Does not support Duplicates**
The structure is unable to store exact copies of the same elements

Let's see how these traits apply to the different data structures we will see in Python...



Lists

Lists

- Lists are **ordered**
- Lists are **mutable**
- Lists **can store duplicates**

```
1 numbers: list[int] = [1, 2, 3, 4, 4]
2
3 # We can loop through a list
4 for number in numbers:
5     print(number, end=" ")
6 print()
7
8 # We can access data at its position
9 print(numbers[2])
10
11 # We add, remove, and modify data
12 numbers.append(5)
13 numbers.pop(0)
14 numbers[2] = 10
15
16 print(numbers)
```

```
1 2 3 4 4
3
[2, 3, 10, 4, 5]
```

Tuples

Tuples

- Tuples are **ordered**
- Tuples are **immutable**
- Tuples **can store duplicates**

```
1 letters: tuple[str] = ["A", "B", "C", "D", "D"]
2
3 # We can loop through a tuple
4 for letter in letters:
5     print(letter, end=" ")
6 print()
7
8 # We can access data at its position
9 print(letters[2])
10
11 # We cannot add, remove, and modify data
12 # letter.append(5) # ❌ Uncomment for error
13 # letter.pop(0)   # ❌ Uncomment for error
14 # letter[2] = 10  # ❌ Uncomment for error
15
16 print(letter)
```

```
A B C D D
C
D
```

Sets

Dictionaries

Dictionaries

- Dictionaries are semi-ordered
- Dictionaries are mutable
- Dictionaries cannot store duplicates

Dictionaries actually store pairs of values, this makes them useful for correlating two pieces of data

Each pair is made up of a key and its value where keys must be unique

```
1 pairs: tuple[float] = {"a": 5, "b": 2} ▶
2
3 # We can loop through a dictionary
4 for item in pairs:
5     print(item, end=" ")
6 print()
7
8 # We can access a value using its key
9 print(pairs["a"])
10
11 # We add, remove, and modify data
12 pairs["c"] = 3
13 pairs.pop("b")
14 pairs["a"] = 1
15
16 print(pairs)
```

```
a b
5
{'a': 1, 'c': 3}
```